TRABAJO FINAL

# Tech Solutions

Aplicación Empresarial en
**Microsoft Visual Studio .NET**

**GRUPO 1:**

Eddy Bryan Gutiérrez Marín
Francisco Castillo Monrroy

PROBLEMÁTICA DEL CASO

# PRÁCTICO PROPUESTO

- La empresa **Tech Solutions** presenta una falta de integración en sus procesos empresariales, lo que genera problemas en la gestión de usuarios, ventas y reportes. Actualmente, **no cuenta con una aplicación** estructurada que permita manejar de forma eficiente la información, provocando **errores en el registro de datos**, inconsistencias en las transacciones y dificultad para generar reportes confiables.

- Además, la ausencia de una arquitectura bien definida limita la escalabilidad del sistema, dificulta el mantenimiento del código y afecta la seguridad, especialmente en la autenticación y autorización de usuarios. **También se identifican problemas en el manejo de concurrencia (hilos), control de transacciones (commit/rollback) y reutilización de componentes.**

# DESCRIPCIÓN GENERAL DEL PROYECTO

1

**Arquitectura N-Capas**

Patrón Singleton para gestión de conexión y configuración

2

**Transacciones SQL**

Manejo de Commit / Rollback en SQL Server y .NET

3

**Reportes**

Generación con parámetros y procedimientos almacenados

4

**Autenticación**

Autorización de usuarios con control de roles

5

**Consultas LINQ**

Acceso a datos con Entity Framework / ADO.NET

6

**Interfaz WPF**

Threads, Timers y bibliotecas DLL reutilizables

# PROPUESTA DE
# SOLUCIÓN Y EVIDENCIAS.

Se propone desarrollar una aplicación empresarial en .NET utilizando arquitectura en N-Capas, lo que permitirá separar responsabilidades (presentación, lógica de negocio y acceso a datos), mejorando la organización, mantenimiento y escalabilidad del sistema.

**La solución incluirá:**

- Implementación de autenticación y autorización de usuarios para garantizar seguridad.
- Uso de ***LINQ*** para consultas eficientes y estructuradas.
- Manejo de transacciones con ***commit y rollback*** para asegurar la integridad de los datos.
- Generación de reportes dinámicos con parámetros mediante procedimientos almacenados.
- Desarrollo de una interfaz moderna con WPF.
- Uso de hilos ***(Threads) y Timers*** para mejorar el rendimiento.
- Aplicación del patrón Singleton para la gestión de conexiones.
- Creación de bibliotecas reutilizables (.dll).
- Elaboración de un instalador ***(Setup Project)*** para facilitar el despliegue

# PROPUESTA DE SOLUCIÓN Y EVIDENCIAS.

## EVIDENCIAS:

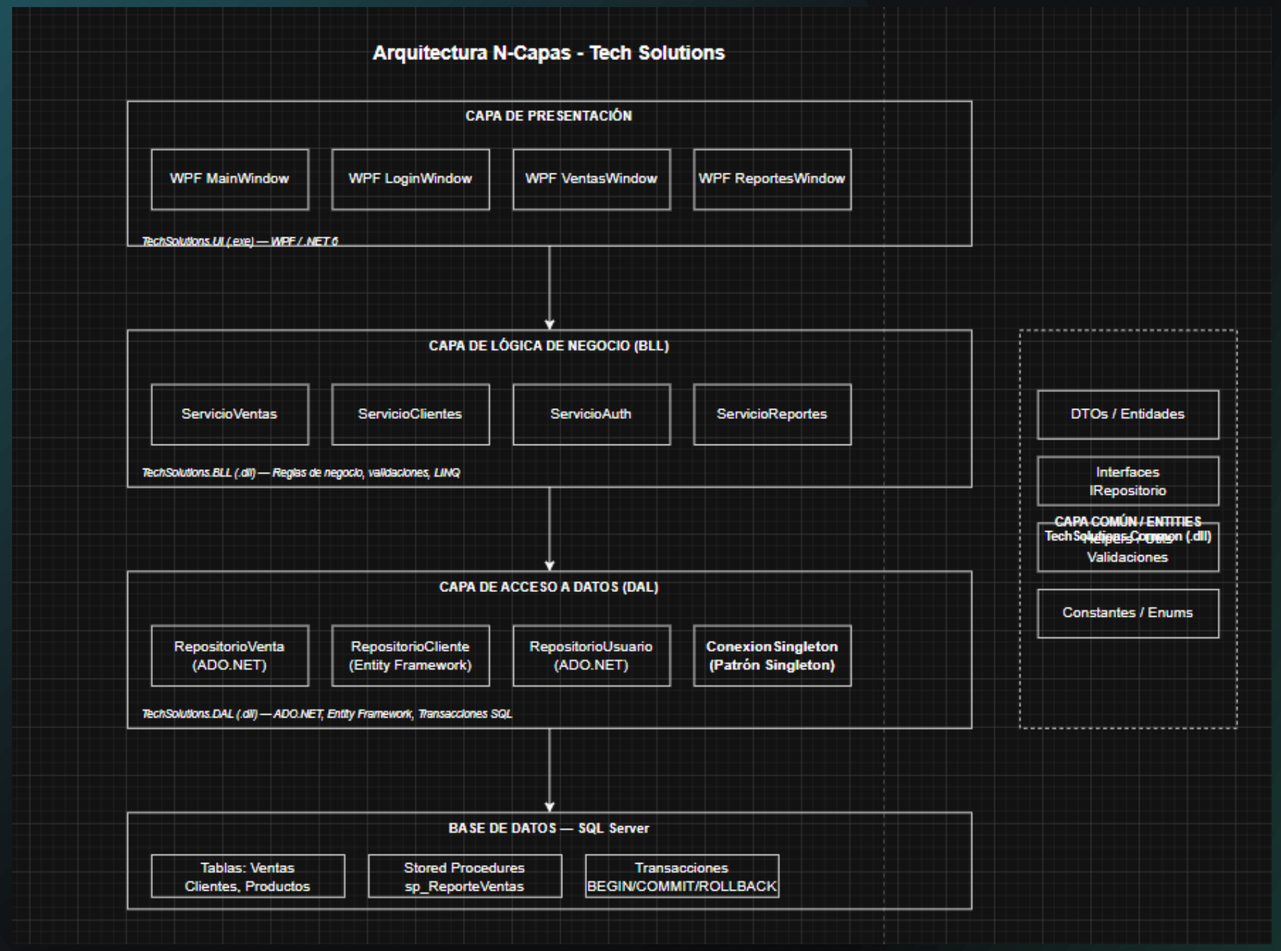
- Diagramas UML ***(N-Capas, clases, secuencia, ER, despliegue).***
- Código fuente estructurado en proyectos.
- ***Scripts SQL*** y procedimientos almacenados.
- Reportes generados en PDF.
- Mockups de la interfaz.
- Pruebas de funcionamiento ***(transacciones, login, reportes).***

# DIAGRAMAS NECESARIOS

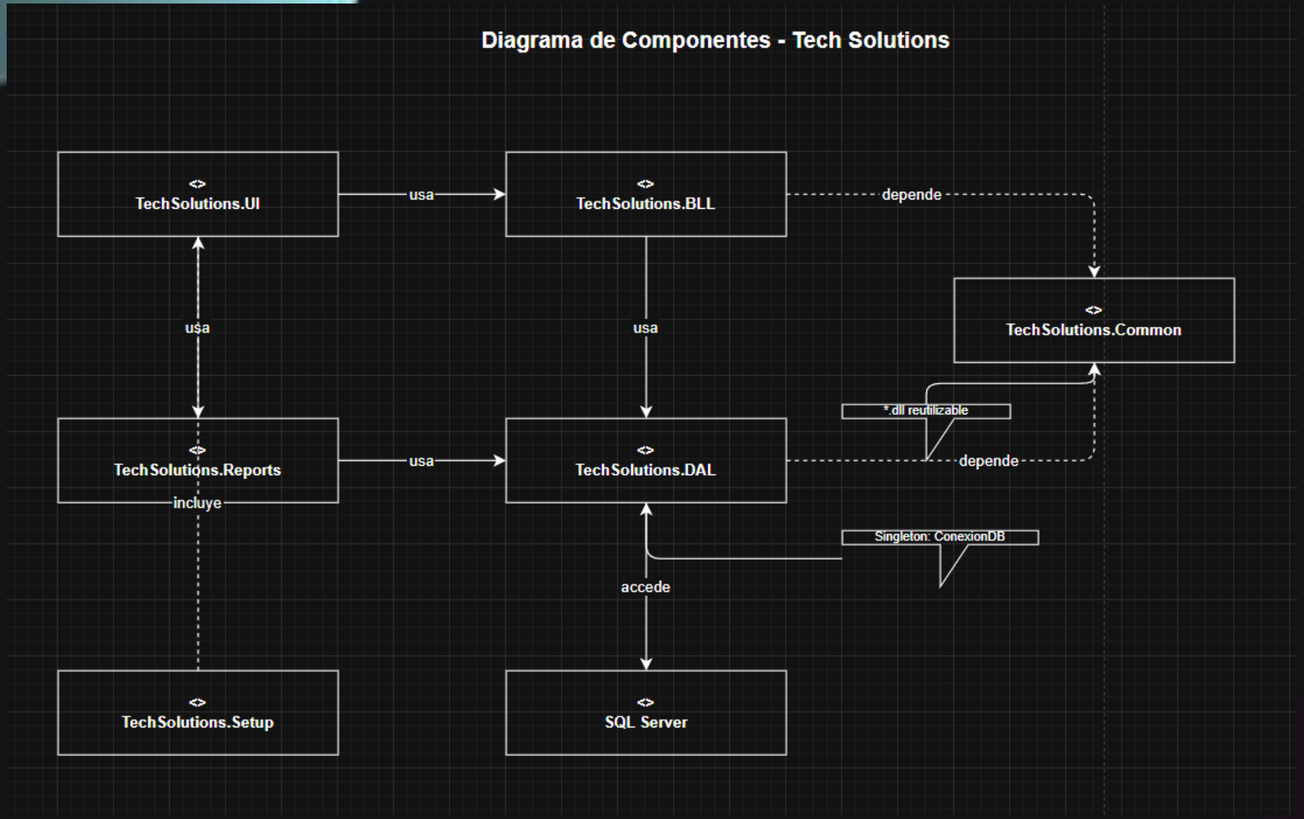
*8 artefactos que respaldan el diseño de la solución*

1. Arquitectura N-Capas
2. Componentes / Packages
3. Clases (Atributos, métodos, relaciones)
4. Entidad–Relación (BD + procedimientos)
5. Secuencia: Venta y Login
6. Actividad: Generación de reportes
7. Despliegue (SQL Server, instalador)
8. Mockups / Wireframes de pantallas

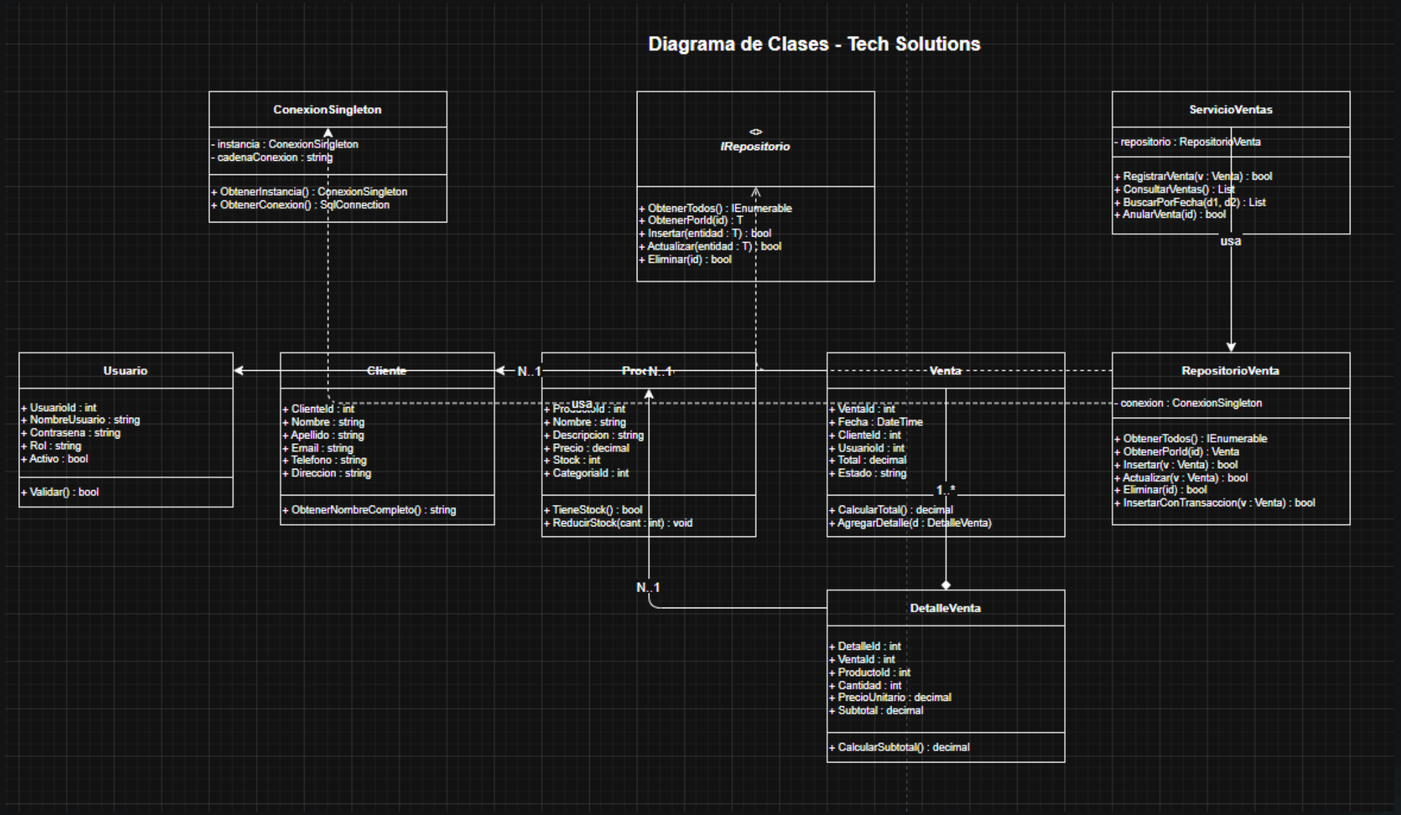
# DIAGRAMA DE ARQUITECTURA



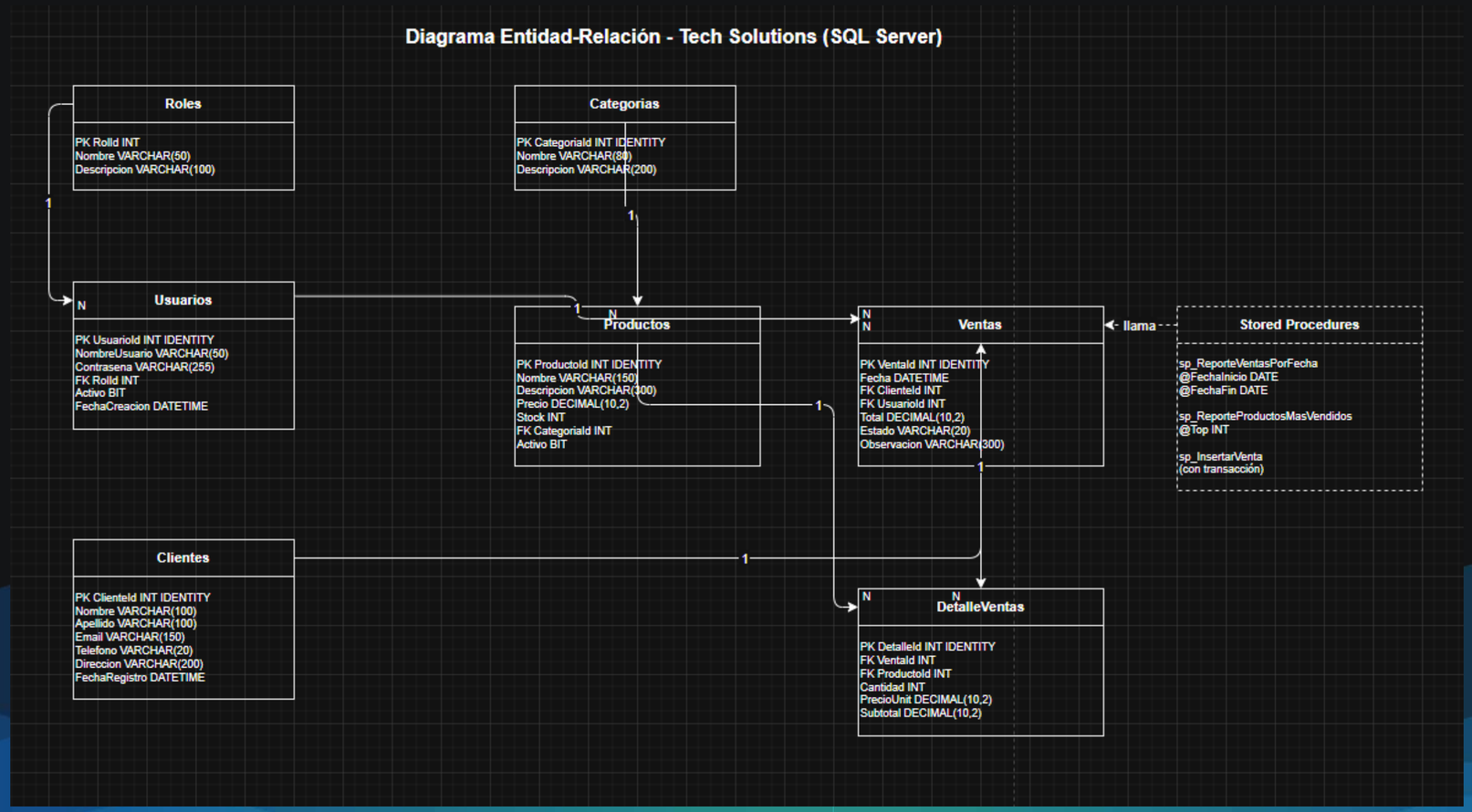
# DIAGRAMA DE COMPONENTES



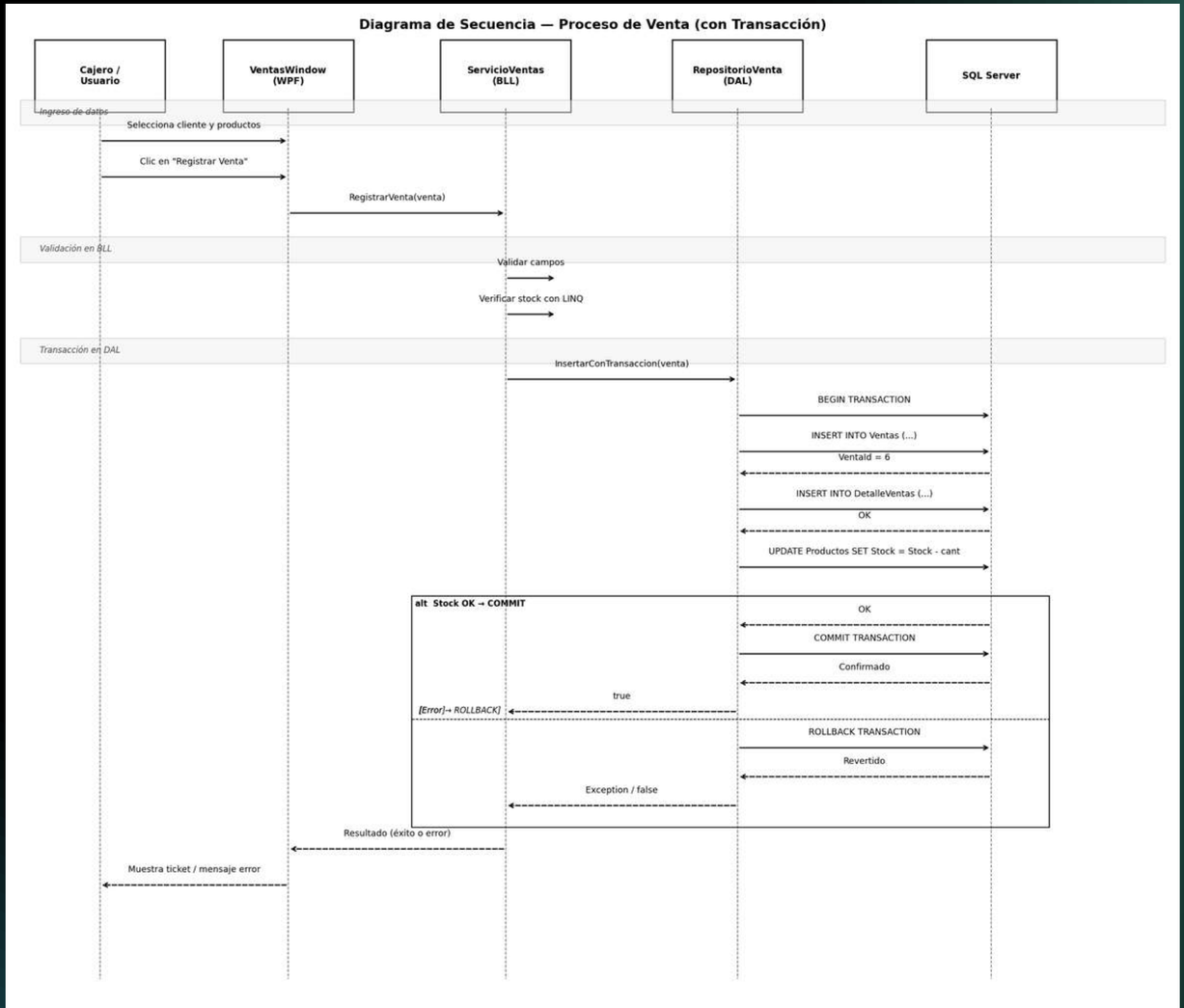
# DIAGRAMA DE CLASES



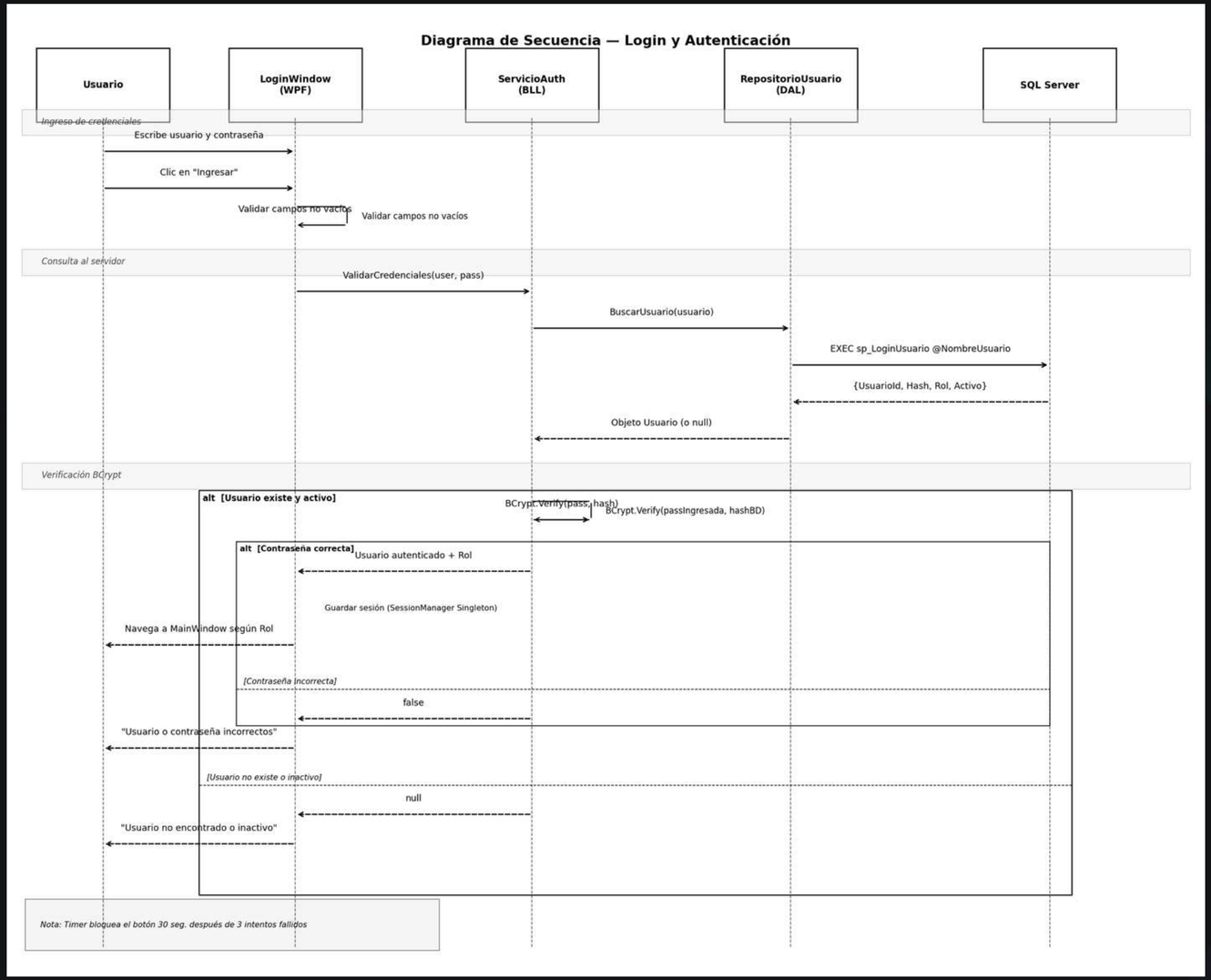
# DIAGRAMA DE ENTIDAD RELACION



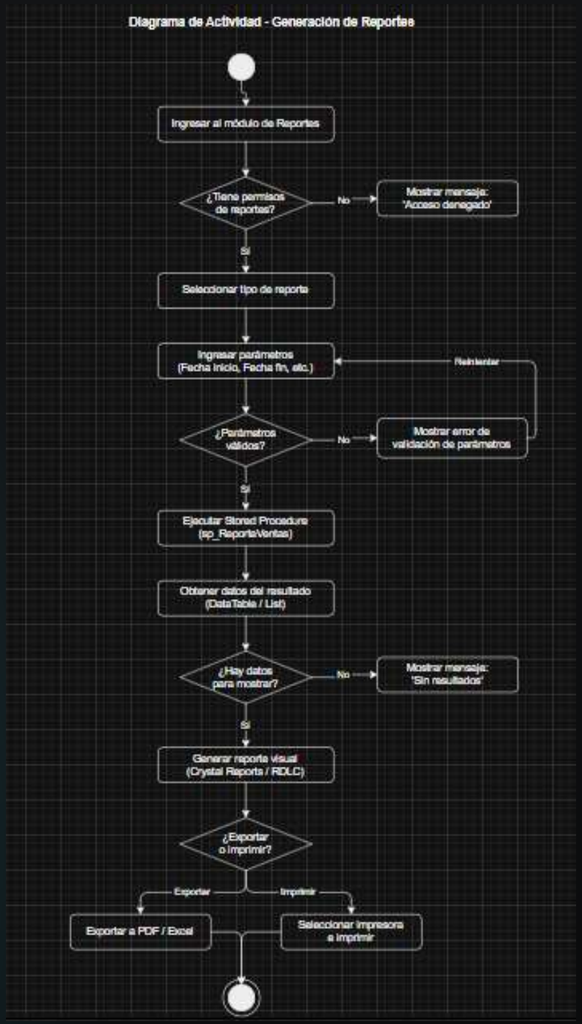
# DIAGRAMA DE SECUENCIA (VENTA)



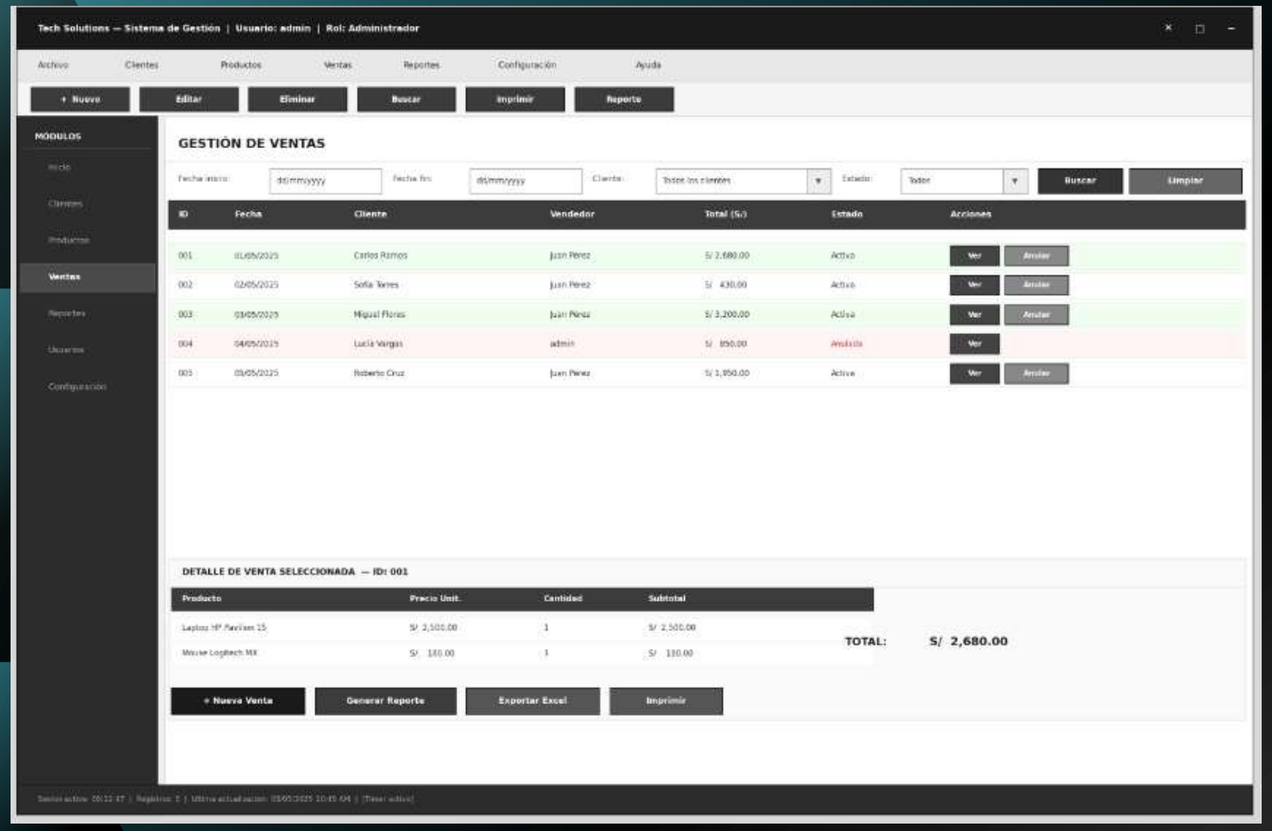
# DIAGRAMA DE SECUENCIA (LOGIN)



# DIAGRAMA DE ACTIVIDAD



# DIAGRAMA DE MOCKUP



# CONCLUSIONES

## Eddy Bryan Gutiérrez Marín

A lo largo de este proyecto comprendí que diseñar una aplicación empresarial va mucho más allá de escribir código que funcione. Implementar la arquitectura N-Capas me obligó a pensar en cada componente de forma aislada, lo que en la práctica significó que un cambio en la base de datos no rompía toda la solución. El patrón Singleton aplicado al gestor de conexión fue un ejemplo concreto de cómo una decisión de diseño correcta evita problemas de rendimiento y concurrencia que, en un sistema real con múltiples usuarios, pueden ser críticos. Me llevo la convicción de que la arquitectura no es un paso opcional que se hace al final, sino la base sobre la que descansa todo lo demás.

## Francisco Javier Castillo Monrroy

Este proyecto me demostró que el desarrollo de software empresarial exige disciplina tanto en la estructura como en la seguridad. Trabajar con transacciones reales en SQL Server, implementando BEGIN, COMMIT y ROLLBACK, me hizo entender que la integridad de los datos es una responsabilidad del desarrollador, no del motor de base de datos por sí solo. Del mismo modo, al diseñar el módulo de autenticación con roles y el manejo de contraseñas con hash, quedó claro que proteger la información del usuario debe ser una decisión tomada desde el primer día del diseño. Salgo de este proyecto con una visión más completa de lo que significa entregar software que no solo cumpla su función, sino que sea robusto, seguro y mantenible en el tiempo.

# Gracias